Secure Coding Review and Vulnerability Assessment

1. Introduction

Secure coding is an essential practice in software development that helps reduce security vulnerabilities in applications. Poor coding practices can expose systems to cyberattacks such as unauthorized access, credential theft, and data breaches.

This project focuses on reviewing an insecure login system developed in Python and identifying security vulnerabilities. A secure version of the code was also implemented by applying secure coding practices.

2. Objective

The main objective of this project is:

- **To identify security vulnerabilities in insecure source code.**
- **To understand the risks associated with poor coding practices.**
- **To implement secure coding techniques for improving application security.**
- **To compare vulnerable and secure coding methods.**

3. Tools and Technologies Used

- **Python Programming Language**
- **Kali Linux Operating System**
- **Terminal / Command Line Interface**
- **Manual Secure Code Review**

4. Vulnerable Code Review

A simple login system was created using Python to demonstrate insecure coding practices.

Vulnerable Code Example

The application uses hardcoded credentials and plain text passwords for authentication.

Security Issues Identified

1. Hardcoded Credentials

The username and password are directly stored in the source code.

Risk:
Attackers can easily access credentials if the source code is exposed.

Recommendation:
Store credentials securely using databases or environment variables.

2. Plain Text Password

The password is stored and compared in plain text format.

Risk:
Passwords can be easily stolen if the system is compromised.

Recommendation:
Use password hashing techniques such as SHA-256 or bcrypt.

3. Weak Authentication Mechanism

The login system only compares static username and password values.

Risk:
The system is vulnerable to brute-force attacks and unauthorized access.

Recommendation:
Implement stronger authentication methods and multi-factor authentication.

4. Lack of Input Validation

User inputs are accepted without validation.

Risk:
Poor input handling can increase security risks.

Recommendation:
Validate and sanitize all user inputs.

~/vulnerable_login.py - Mousepad

File Edit Search View Document Help

```python
username = input("Enter Username: ")
password = input("Enter Password: ")

if username == "admin" and password == "admin123":
    print("Login Successful")
else:
    print("Invalid Login")

```

blackking@kali: ~

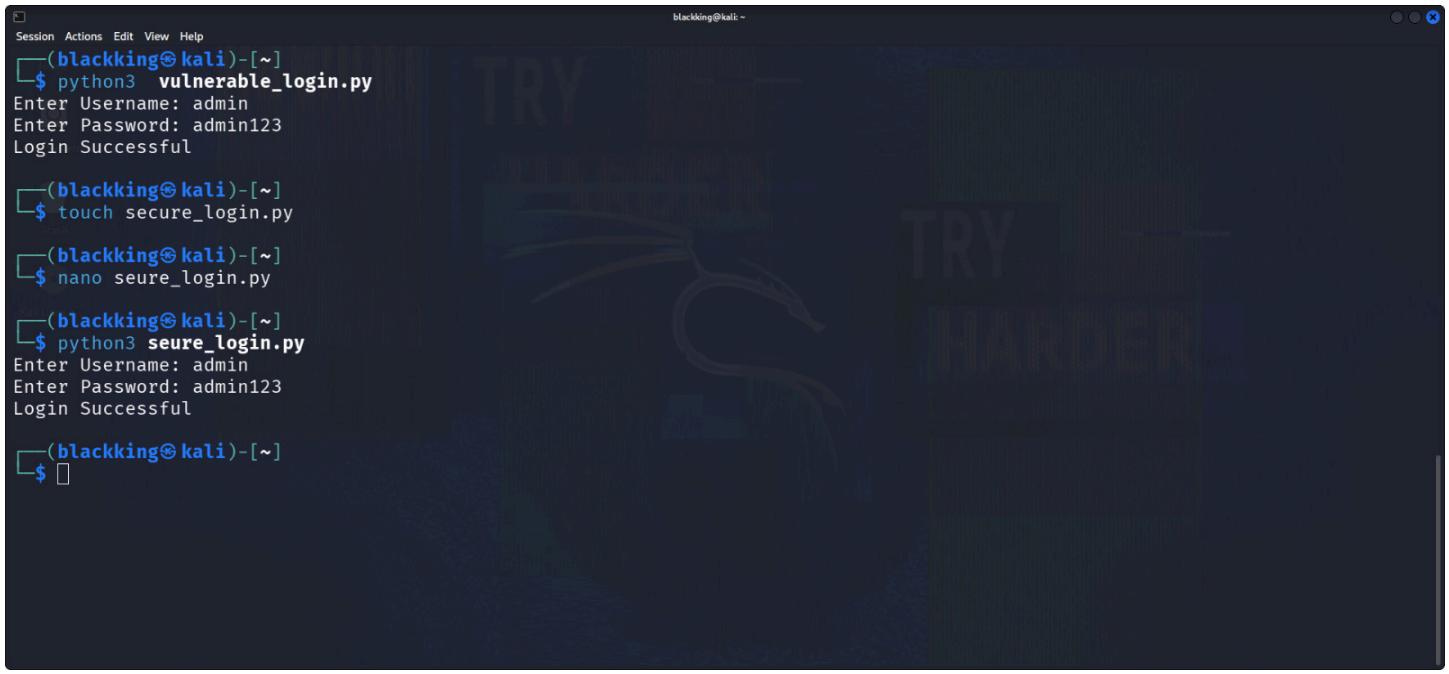
Session Actions Edit View Help

```
┌──(blackking㉿kali)-[~]
└─$ python3 vulnerable_login.py
Enter Username: admin
Enter Password: admin123
Login Successful

┌──(blackking㉿kali)-[~]
└─$
```

~/seure_login.py - Mousepad

File Edit Search View Document Help

```python
import hashlib

username = input("Enter Username: ")
password = input("Enter Password: ")

stored_username = "admin"

stored_password = hashlib.sha256(
    "admin123".encode()
).hexdigest()

entered_password = hashlib.sha256(
    password.encode()
).hexdigest()

if username == stored_username and entered_password == stored_password:
    print("Login Successful")
else:
    print("Invalid Login")

```



5. Secure Code Implementation

To improve security, a secure version of the login system was developed.

Security Improvements Applied

- **Password hashing using SHA-256**
- **Better authentication logic**
- **Improved password handling**
- **Reduced credential exposure**

These improvements help strengthen application security and reduce vulnerabilities

6. Output

The vulnerable and secure versions of the login system were successfully tested using the terminal.

The vulnerable version demonstrated insecure coding practices, while the secure version showed improved password handling and authentication security.

7. Learning Outcome

Through this project, I learned:

- **The importance of secure coding practices**
- **How to identify software vulnerabilities**
- **Password hashing techniques**
- **Authentication security concepts**
- **Best practices for secure software development**

8. Conclusion

Secure coding plays an important role in protecting applications from cyber threats and vulnerabilities. This project helped in understanding common coding mistakes and how to implement secure practices to improve software security.

By replacing insecure coding methods with secure authentication techniques, the application became more resistant to security risks and attacks.